

CSC1029 OO Programming

Lecture 22 – Searching Sorted Data



What's to come in today's lecture?

- Searching Sorted Data
- Binary Search
- The Need for Sorting
- An introduction to Insertion Sort
- Some useful String methods

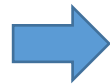
Searching for Information (Again!)

How do you look for something in a telephone directory?

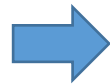
What *input* is required for the search process?

What *output* is produced from the search process?

Search Criteria

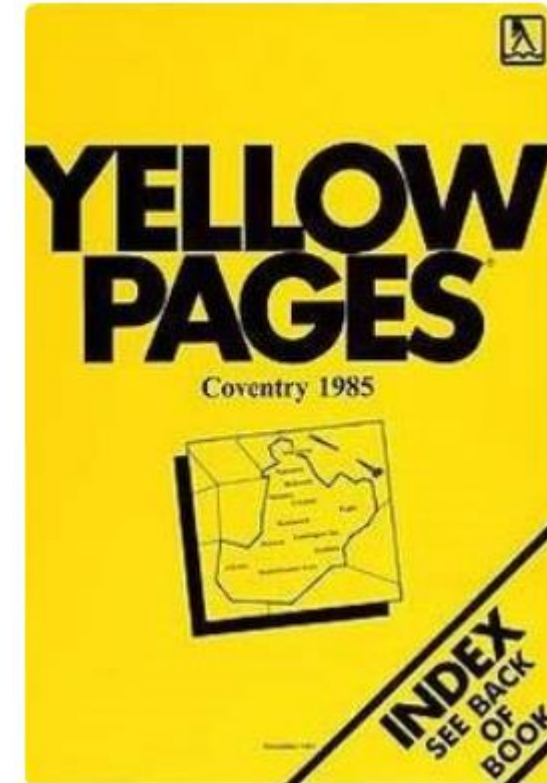


Search
Algorithm



Search Result

Data to Search



Searching for Information (Again!)

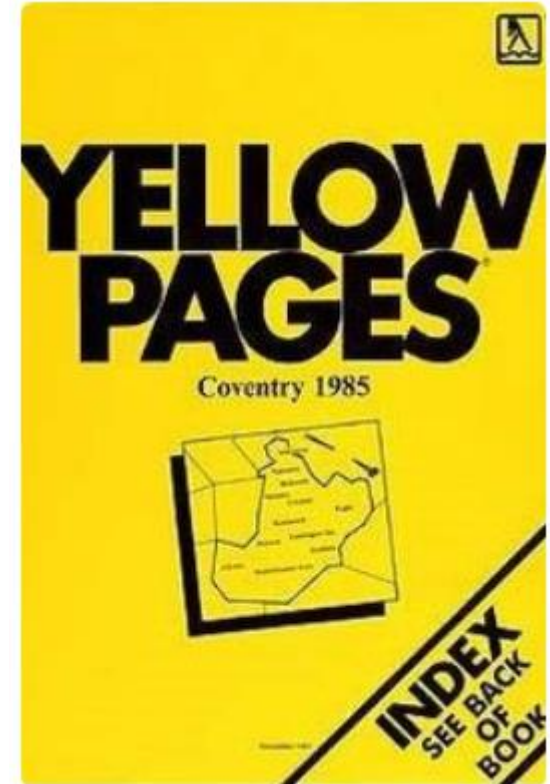
For a **linear** (sequential) search – data can be in any order.

What difference would it make (for searching) if the data was ordered?

As was demonstrated in the last lecture, it's quicker to locate indexed information (e.g. in a book) than it would be if sequential searching was all we had.

Why is this the case?

We can use the fact that data is organised (**sorted**) to reduce the number of comparisons and more rapidly 'home in' on the search target (or more rapidly work out that it's not to be found).



Searching for Information (Again!)

Binary Search is an algorithm that *takes advantage of data order*.

```
int data[] = {1, 3, 6, 8, 9, 11, 14, 16, 20}; // in ascending order
```

0	1	2	3	4	5	6	7	8
1	3	6	8	9	11	14	16	20

Example **search cases** to consider ...

Value at the start – e.g. 1

Value at the end – e.g. 20

Value in the middle – e.g. 9

Value not in array – e.g. 10

Searching for Information (Again!)

A Binary Search algorithm:

start = 0

end = last index in array

repeat

 pivot = start + (end-start)/2

 if target == value at pivot then

 return pivot

 if value at pivot > target

 end = pivot - 1

 else

 start = pivot + 1

while start <= end

0	1	2	3	4	5	6	7	8
1	3	6	8	9	11	14	16	20



start

target : 14

start : 0

end :

pivot :

Searching for Information (Again!)

A Binary Search algorithm:

start = 0

end = last index in array

repeat

 pivot = start + (end-start)/2

 if target == value at pivot then

 return pivot

 if value at pivot > target

 end = pivot - 1

 else

 start = pivot + 1

while start <= end

0	1	2	3	4	5	6	7	8
1	3	6	8	9	11	14	16	20
↑								↑
start								end

target : 14

start : 0

end : 8

pivot :

Searching for Information (Again!)

A Binary Search algorithm:

```
start = 0
end = last index in array
repeat
    pivot = start + (end-start)/2
    if target == value at pivot then
        return pivot
    if value at pivot > target
        end = pivot - 1
    else
        start = pivot + 1
while start <= end
```

0	1	2	3	4	5	6	7	8
1	3	6	8	9	11	14	16	20
↑				↑				↑
start				pivot				end

target : 14

start : 0

end : 8

pivot : 4

Searching for Information (Again!)

A Binary Search algorithm:

```
start = 0
end = last index in array
repeat
    pivot = start + (end-start)/2
    if target == value at pivot then
        return pivot
    if value at pivot > target
        end = pivot - 1
    else
        start = pivot + 1
while start <= end
```

0	1	2	3	4	5	6	7	8
1	3	6	8	9	11	14	16	20
↑				↑				↑
start				pivot				end

target : 14 target(14) == pivot value(9)

start : 0

end : 8

pivot : 4



Searching for Information (Again!)

A Binary Search algorithm:

```
start = 0
end = last index in array
repeat
    pivot = start + (end-start)/2
    if target == value at pivot then
        return pivot
    if value at pivot > target
        end = pivot - 1
    else
        start = pivot + 1
while start <= end
```

0	1	2	3	4	5	6	7	8
1	3	6	8	9	11	14	16	20
↑				↑				↑
start				pivot				end

target : 14 pivot value(9) > target(14)

start : 0

end : 8

pivot : 4



Searching for Information (Again!)

A Binary Search algorithm:

```
start = 0
end = last index in array
repeat
    pivot = start + (end-start)/2
    if target == value at pivot then
        return pivot
    if value at pivot > target
        end = pivot - 1
    else
        start = pivot + 1
while start <= end
```

0	1	2	3	4	5	6	7	8
1	3	6	8	9	11	14	16	20
				↑	↑			↑
				pivot start				end

target : 14

start : 5

end : 8

pivot : 4

Searching for Information (Again!)

A Binary Search algorithm:

```
start = 0
end = last index in array
repeat
    pivot = start + (end-start)/2
    if target == value at pivot then
        return pivot
    if value at pivot > target
        end = pivot - 1
    else
        start = pivot + 1
while start <= end
```

0	1	2	3	4	5	6	7	8
1	3	6	8	9	11	14	16	20
				↑	↑			↑
				pivot start				end

target : 14 start <= end
start : 5
end : 8
pivot : 4



Searching for Information (Again!)

A Binary Search algorithm:

```
start = 0
end = last index in array
repeat
    pivot = start + (end-start)/2
    if target == value at pivot then
        return pivot
    if value at pivot > target
        end = pivot - 1
    else
        start = pivot + 1
while start <= end
```

0	1	2	3	4	5	6	7	8
1	3	6	8	9	11	14	16	20
					↑	↑		↑
					start	pivot		end

target : 14

start : 5

end : 8

pivot : 6

Searching for Information (Again!)

A Binary Search algorithm:

```
start = 0
end = last index in array
repeat
    pivot = start + (end-start)/2
    if target == value at pivot then
        return pivot
    if value at pivot > target
        end = pivot - 1
    else
        start = pivot + 1
while start <= end
```

0	1	2	3	4	5	6	7	8
1	3	6	8	9	11	14	16	20

↑ ↑ ↑
start pivot end

target : 14 target(14) == pivot value(14)
start : 5
end : 8
pivot : 6



Searching for Information (Again!)

```
public static int locationOf(int target, int data[])
{
    int start = 0;
    int end = data.length-1;
    int pivot;
    do {
        pivot = start + ((end-start)/2);
        if (data[pivot] == target) {
            return pivot;
        }
        if (data[pivot] > target) {
            end = pivot-1;
        }
        else {
            start = pivot+1;
        }
    }
    while(start <= end);
    return -1;
}
```



```
public static void main(String args[]) {
    Scanner input = new Scanner(System.in);
    int data[] = createRandomOrderedArray(11);
    int target;
    display(data);
    do {
        System.out.print("Enter value:");
        target = input.nextInt();
        if (target != -1) {
            int pos = locationOf(target, data);
            if (pos >= 0) {
                System.out.println(target + " found at "
                                   + pos);
            }
            else {
                System.out.println(target + " not
                                   found.");
            }
        }
    }
    while(target != -1);
    input.close();
}
```

Searching for Information (Again!)

```
[4, 16, 20, 43, 58, 77, 86, 88, 92, 95, 96]
Enter value:4
4 found at 0
Enter value:96
96 found at 10
Enter value:16
16 found at 1
Enter value:5
5 not found.
Enter value:-1
```

```
public static void main(String args[]) {
    Scanner input = new Scanner(System.in);
    int data[] = createRandomOrderedArray(11);
    int target;
    display(data);
    do {
        System.out.print("Enter value:");
        target = input.nextInt();
        if (target != -1) {
            int pos = locationOf(target, data);
            if (pos >= 0) {
                System.out.println(target + " found at "
                                   + pos);
            }
            else {
                System.out.println(target + " not
                                   found.");
            }
        }
    }
    while(target != -1);
    input.close();
}
```

It All Depends on Sorted Data ...

The Binary Search algorithm depends on sorted data.

[4, 16, 20, 43, 58, 77, 86, 88, 92, 95, 96]

What options do we have here?

The main method for the previous example used a method to produce a sorted array:

```
int data[] = createRandomOrderedArray(11);
```

This method could use a number of different algorithms to produce the desired result.

Bubble Sort

Quick Sort

These algorithms will take unsorted data as input and produce a sorted result.

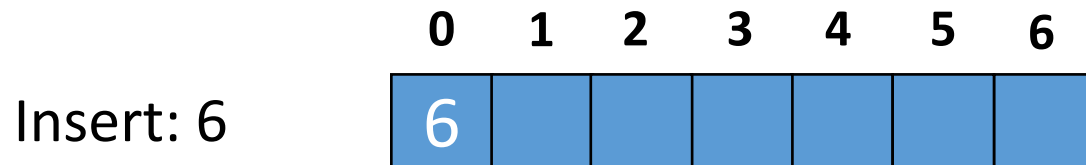
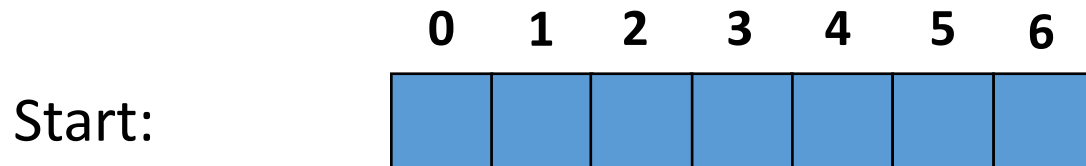
We will examine these algorithms and their application another time.

createRandomOrderedArray does not use the above techniques ...

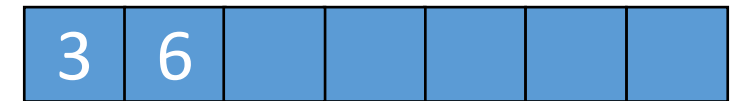
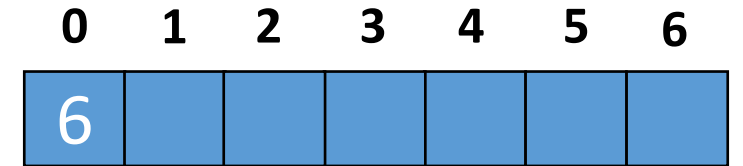
It All Depends on Sorted Data ...

`createRandomOrderedArray` starts with an empty array, creating and inserting elements in their correct location.

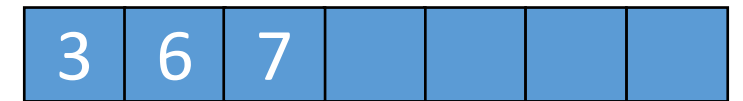
This technique is called ***Insertion Sort***.



Insert: 3



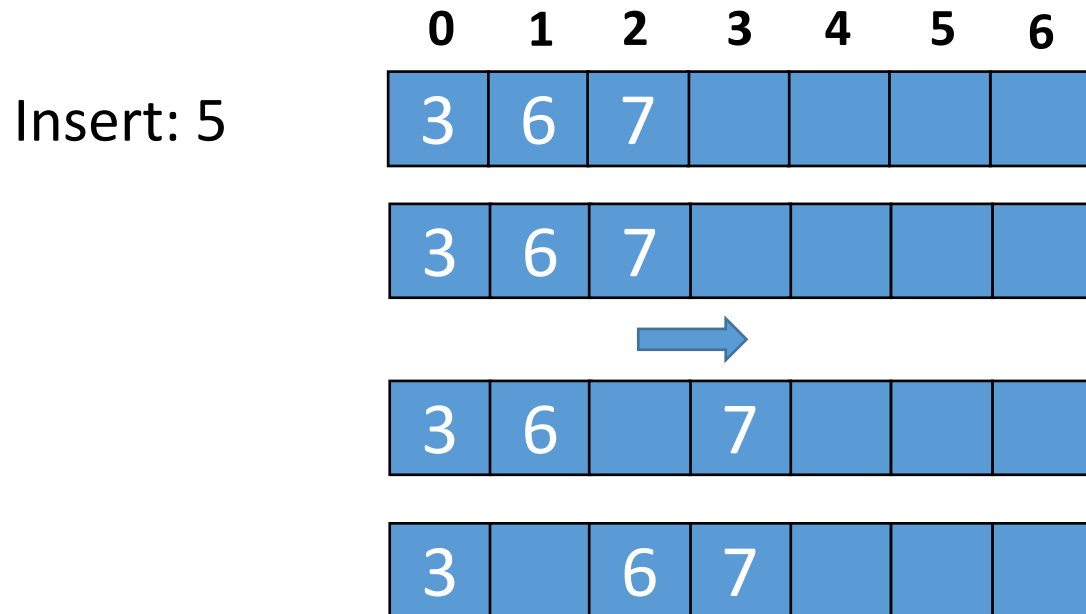
Insert: 7



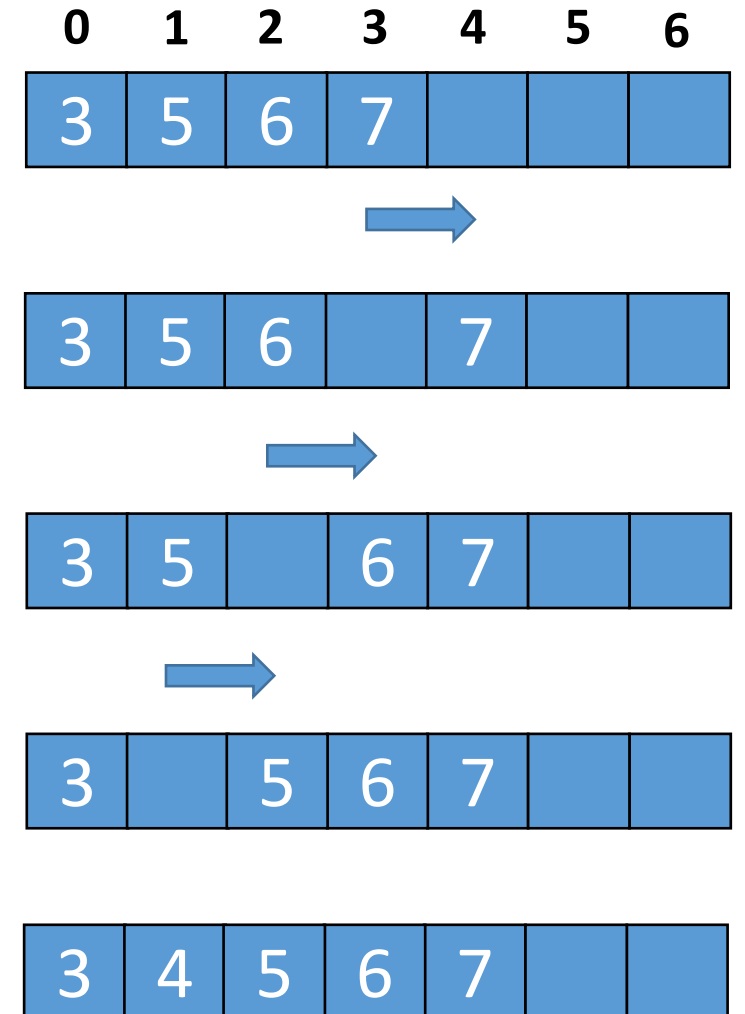
It All Depends on Sorted Data ...

`createRandomOrderedArray` starts with an empty array, creating and inserting elements in their correct location.

This technique is called ***Insertion Sort***.



Insert: 4



Some Useful String Methods

The String class has a range of useful methods ...

```
// Comparing Strings
```

```
if ( str1.equals( str2 ) ) {  
    System.out.println(str1 + " is equal to " + str2);  
}  
else {  
    System.out.println(str1 + " is different from " + str2);  
}
```

Some Useful String Methods

The String class has a range of useful methods ...

// Comparing Strings

```
if ( str1.compareTo( str2 ) == 0 ) {  
    System.out.println(str1 + " is equal to " + str2);  
}  
else if ( str1.compareTo( str2 ) < 0 ) {  
    System.out.println(str1 + " comes before " + str2);  
}  
else {  
    System.out.println(str1 + " comes after " + str2);  
}
```

Some Useful String Methods

The String class has a range of useful methods ...

// Chopping up a String

```
String fullName = "Simpson, Homer, Jay";  
String names[] = fullName.split(",");
```

```
0    "Simpson"  
1    "Homer"  
2    "Jay"
```

Some Useful String Methods

The String class has a range of useful methods ...

```
// Removing white space characters at either end
String fullName = "    CSC1027 Programming    ";
fullName = fullName.trim();
```

```
// fullName will contain "CSC1027 Programming"
// after trim
```



What's Next?

Sorting Data

A closer look at Insertion Sort

An overview of Software Testing